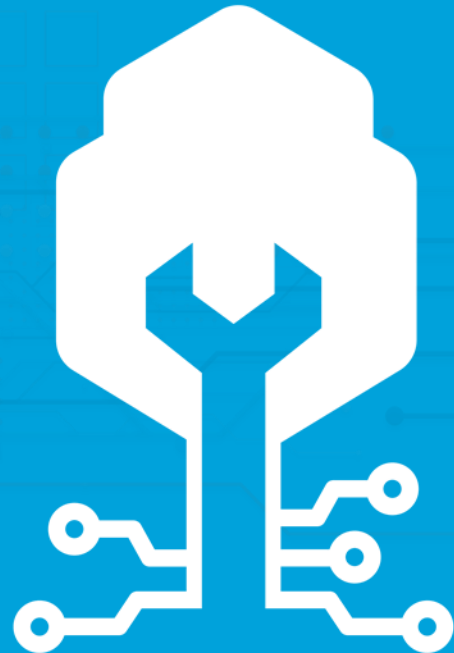


Databases

KEN2110

Denise Slenter

Adapted with gratitude from the original lectures by Dr. Ashish Sai, Dr. Katharina Schneider, Dr. Max Sondag, and dr. Tom Pepels



What have we looked at so far?

- Database
 - Collection of logically related data
 - Two formal definitions
- Database Management System
 - Program used for defining and maintaining this database.
- Data Models
 - Conceptual, Logical and Physical Data Models
- Relational Data Model
 - An overview of entity and relationship model
- SQL as data definition language
 - Creating and altering tables

Learning Objectives

- Understand SQL Fundamentals
 - Learn the basic functions of **SQL as a data manipulation language**, including the structure and execution of queries using SELECT, FROM, and WHERE clauses.
- Query Formulation and Execution
 - Develop skills in crafting **effective SQL queries** to manipulate and retrieve data, understanding **how to apply conditions, expressions, and patterns** to refine results
- Database normalization (fundamentals)
 - Why should we apply **normalization**?
 - Which normalization **levels** are available?

SQL (Part 2)

SQL as Data Manipulation Language



SELECT-FROM-WHERE Statements

```
SELECT desired attributes  
FROM one (or more) tables  
WHERE condition about tuples of the table(s);
```

- The where clause is optional

Example

- Using relation Member: find the last names of the members whose role is “Soldier”

```
SELECT last_name  
FROM Member  
WHERE role = 'Soldier';
```

- The answer is a relation with a single attribute – last_name – and tuples (rows) with the last_name of each Soldier

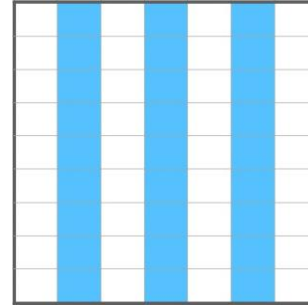
last_name
Jacobs
Kusters
Peeters

Formal Semantics: Single-Relation Query

- Begin with the relation in the FROM clause
- Apply the *selection* indicated by the WHERE clause
- Apply the *projection* indicated by the SELECT clause



Selection



Projection

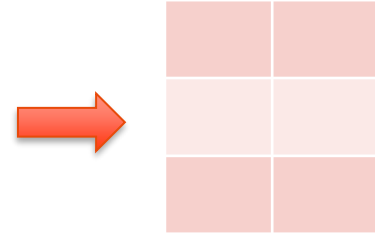
Operational Semantics: Single-Relation Query

- Think of a tuple variable visiting each tuple of the relation mentioned in FROM
- Check if the “current” tuple satisfies the WHERE clause
- If so, compute the attributes or expressions of the SELECT clause using the components of this tuple



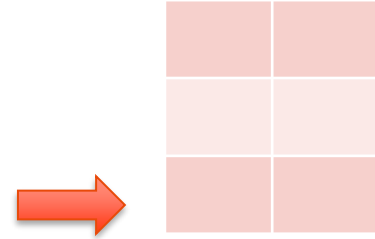
Operational Semantics: Single-Relation Query

- Think of a tuple variable visiting each tuple of the relation mentioned in FROM
- Check if the “current” tuple satisfies the WHERE clause
- If so, compute the attributes or expressions of the SELECT clause using the components of this tuple



Operational Semantics: Single-Relation Query

- Think of a tuple variable visiting each tuple of the relation mentioned in FROM
- Check if the “current” tuple satisfies the WHERE clause
- If so, compute the attributes or expressions of the SELECT clause using the components of this tuple



SELECT * clause

- When there is one relation in the FROM clause, * in the SELECT clause stands for “all attributes of this relation”

Example:

```
SELECT *  
FROM Member  
WHERE role = 'Soldier';
```

member_id	alias	first_name	last_name	role	join_date	status	phone
1005	Mergel	Sam	Jacobs	Soldier	2022-09-01	active	06-10000006
1006	Tunnel	Eva	Kusters	Soldier	2023-01-17	active	06-10000007
1007	Maas	Daan	Peeters	Soldier	2021-10-01	active	06-10000008

Renaming attributes in results

- If you want the results to have different attribute names, use *AS* to rename an attribute

Example:

```
SELECT last_name AS surname  
FROM Member  
WHERE role = 'Soldier';
```

surname
Jacobs
Kusters
Peeters

Expressions in SELECT Clauses

- Any expression that makes sense can appear as an element of a SELECT clause
- Example

```
SELECT entry_ID, amount,  
       amount*1.09 AS amountUSD  
FROM Financial_entry;
```

entry_ID	amount	amountUSD
8012	600.00	654.00
8013	350.00	381.50
8014	520.00	566.80
8015	780.00	850.20
8016	260.00	283.40
8017	1100.00	1199.00
8018	4200.00	4578.00
8019	290.00	316,10
8020	150.00	163,50
8021	2100.00	2289,00
8022	980.00	1068,20
8023	430.00	468,70
8024	600.00	654,00
8025	200.00	239,80
8026	1700.00	1853,00

Constants as Expressions in SELECT Clause

```
SELECT entry_id, amount*1.09 AS  
       received, 'USD' AS  
       currency  
FROM financial_entry
```

entry_ID	received	currency
8012	654.00	USD
8013	381.50	USD
8014	566.80	USD
8015	850.20	USD
8016	283.40	USD
8017	1199.00	USD
8018	4578.00	USD
8019	316,10	USD
8020	163,50	USD
8021	2289,00	USD
8022	1068,20	USD
8023	468,70	USD
8024	654,00	USD
8025	239,80	USD
8026	1853,00	USD

Complex Conditions in WHERE Clause

- Boolean operators AND, OR, NOT
- Comparisons =, <>, <, >, <=, >=
- <> is equivalent to != in MySQL (means NOT EQUAL)
- OR is equivalent to || in MySQL
- AND is equivalent to && in MySQL
- And many other operators that produce Boolean-valued results
- <https://dev.mysql.com/doc/refman/8.4/en/logical-operators.html>

Example: Complex Conditions in WHERE Clause

- Using relation financial_entry , find the entry_ids of the transfers with amount greater or equal to 600.00 and transferred via “bank transfer”

```
SELECT entry_id
FROM financial_entry
WHERE amount >= 600.00
      AND payment_method = 'bank transfer';
```

entry_id
8018
8021
8026

IS NULL

```
SELECT alias, first_name  
FROM Member  
WHERE last_name = NULL;
```

```
SELECT alias, first_name  
FROM Member  
WHERE last_name is NULL;
```

alias	first_name
-------	------------

= **NULL never works.** NULL in SQL means *unknown* or *no value* and comparing anything to an unknown using = produces NULL (not TRUE or FALSE). So, the condition is never satisfied and no rows are returned — even rows that have a NULL value in that column

alias	first_name
Berg	Nina
Sluier	Iris
Dossier	Rosa

Patterns in WHERE Clause

- A condition can compare a string to a pattern by
 - <Attribute> LIKE <pattern>or
 - <Attribute> NOT LIKE <pattern>
- Pattern is a quoted string
- Wildcards in LIKE
 - %: matches any sequence of characters (string)
 - _: matches exactly one character

Example 1: LIKE in WHERE clause

- Using relation Member, find the names of all Soldiers that have 'ers' in their last name

```
SELECT first_name, last_name
FROM Member
WHERE role = 'Soldier' AND last_name LIKE '%ers%';
```

first_name	last_name
Eva	Kusters
Daan	Peters

Example 2: LIKE in WHERE clause

- Using relation Members, find all details about members who joined in December (any year)

```
SELECT *  
FROM Member  
WHERE join_date LIKE '____-12%';
```

- Watch out, use exactly 4 times underline sign!!



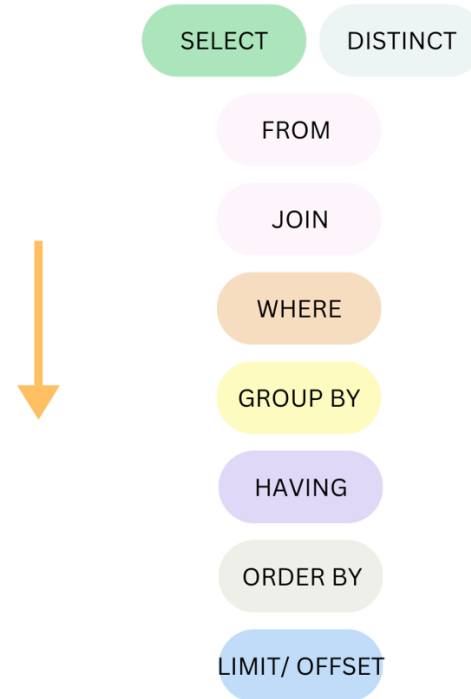
- Remember, datatype for Dates and Times:
 - DATE
 - form of a date value `yyyy-mm-dd`

SQL queries order

Discussed in Chapter 3 of
A.Beaulieu, Learning SQP, 2020
(3rd edition), O'Reilly

**Except for JOIN (next lecture)
**Except LIMIT (tutorial 1
Notebook)

Order of Writing SQL Queries



Missing and Erroneous Data



What can go wrong with data?

Missing Data	No measurements, ...?
Erroneous Values	Misspelling, outliers, ...?
Type Conversion	Date formats (01-03-2025 vs 2025/03/01)
Entity Resolution	M. Bellini = Marco Bellini?

Erroneous Data

- Values are present, but wrong, inconsistent or implausible:
 - Misspelling: 'Soldier' vs. 'Solder'
 - Inconsistent Capitalization: 'active' vs 'Active'
 - Wrong value (but valid type): expected_amount = -500
 - Outliers: expected_amount = 9999999999999999
- Prevention:
 - Controlled input (instead of free text use dropdown, radio buttons, ...)
 - Use predefined domains (e.g. lookup tables)
 - Standardization rules (lowercase everything, trim spaces, ...)
- Handling:
 - Manual cleaning
 - `UPDATE member`
`SET alias = TRIM(LOWER(alias));`

Type Conversion

- Same information, but stored in different formats or types
 - Dates: '01-03-2025' vs '2025/03/01'
 - Numbers stored as text: '1000' vs 1000
 - ...
- Prevention
 - Use correct data types in your DB (DATE, DECIMAL for numbers you can do calculations with)
 - Define consistent formats as input level (e.g. always use YYYY-MM-DD)
 - Be explicit about units
- Handling
 - Convert to a standard format
 - e.g. UPDATE operation SET date_column = STR_TO_DATE(date_column, '%d-%m-%Y');
 - Normalize units

“Normalizing” Units in SQL

- No native unit system, but you can:
- Encode unit in column name
 - `'length_minutes SMALLINT'`
- Only store one canonical unit with documentation
 - `-- length stored in minutes`
`'length SMALLINT,'`



Remember, this is a comment in SQL

“Normalizing” Units in SQL

- No native unit system, but you can:
- Encode unit in column name
 - `'length_minutes SMALLINT'`
- Only store one canonical unit with documentation
 - `-- length stored in minutes`
`'length SMALLINT,'`
- Add a unit attribute (column)
 - `CREATE TABLE measurement (
 measurement_id INT PRIMARY KEY,
 value DECIMAL(10,4),
 unit VARCHAR(20) NOT NULL -- e.g. 'kg', 'lb', 'mg'`
`);`
- Use CHECK constraint to guard ranges
 - `length_minutes SMALLINT CHECK (length_minutes BETWEEN 1 AND 600),`



Remember, this is a comment in SQL

“Normalizing” Units in SQL

- No native unit system, but you can:
- Encode unit in column name
 - `'length_minutes SMALLINT'`
- Only store one canonical unit with documentation
 - `-- length stored in minutes`
`'length SMALLINT,'`
- Add a unit attribute (column)
 - `CREATE TABLE measurement (
 measurement_id INT PRIMARY KEY,
 value DECIMAL(10,4),
 unit VARCHAR(20) NOT NULL -- e.g. 'kg', 'lb', 'mg'`
);
- Use CHECK constraint to guard ranges
 - `length_minutes SMALLINT CHECK (length_minutes BETWEEN 1 AND 600),`



Remember, this is a comment in SQL



Question: when would you use which approach? ~2 minutes

Entity Resolution

- Different records could refer to the same real-world entity
 - (1001, 'Shadow', Marco Bellini) vs. (1055, 'Shadow', M. Bellini)
 - Maastricht vs. Maastrich (erroneous value?)
 - 'The Boss' vs 'Boss'
- Prevention:
 - Use stable identifiers (e.g. IDs rather than names)
 - Use keys and uniqueness constraints
 - Reuse existing records (let users choose from an existing list)
 - Search before insert (-> Suggestion by System: "did you mean...")
- Handling
 - Compare records carefully
 - Manual merging (comparing records)

Conclusion

- Handling data issues after inserting the data is costly
- Make sure to have a good system to insert the data

Missing Values - NULL

- Tuples in SQL relations can have NULL as a value for one or more components

Concept	R	Python	Java	SQL
Missing / unknown value	NA	None	null	NULL
Not a number (0/0)	NaN	float('nan')	Double.NaN	NULL
Missing float	NA_real_	float('nan')	null (boxed) / Double.NaN	NULL
Missing string	NA_character_	None	null	NULL
Undefined result	NaN / Inf	nan / inf	Double.NaN / Double.POSITIVE_INFINITY	NULL

Missing Values - NULL

- Tuples in SQL relations can have NULL as a value for one or more components
- Meaning depends on context. Three common cases:
 - Missing value: e.g., we know a member has some address, but we do not know what it is
 - Inapplicable: e.g., the value of attribute name of spouse for an unmarried person
 - Not yet known: will be filled later (e.g. delivery date)

Should this be NULL at all?

- NULL often signals a modeling problem, not just missing data
- Prevention: How can you avoid NULL values?
 - Use NOT NULL as default values
 - E.g. status = 'pending' instead of NULL
 - Use explicit flags
 - E.g. has_last_name BOOLEAN
 - Split into separate tables
 - E.g. optional information -> new relation

Short Break

- **10 minutes**



Database Normalization



Databases != automatic mirrors

- Reality changes constantly, but databases only change when we record updates.
- If updates are late or incorrect → a gap opens between data and reality.
- Normalization limits inconsistencies, but discipline and process are essential.
- The database is not truth itself, but a maintained model of truth

Databases need to be accurate

- Databases aren't just storage → they model reality.
- With that comes the responsibility for accuracy.
- Normalization reduces redundancy and enforces integrity.
- Goal: a single source of truth that mirrors the real world.

What is normalization?

- Eliminate redundancy in database design.
- Redundancy → trust issues + hard-to-update data.
- Without normalization → frequent Insertion, Update, and Deletion anomalies.
- Approach: split tables into smaller, related ones until anomalies are minimized.

Why is normalization important to YOU

- As a data scientist:
 - You will come across a lot of data in databases, understanding how data is normalized helps you find the structure of the database faster.
 - You should be able to discuss with others (developers, DBA's) on how to best structure data
- As a developer:
 - Understanding database structure to be able to create ACID databases
 - Modelling data is a part of the development process

Why is normalization important to YOU

- As a data scientist:
 - You will come across a lot of data in databases, understanding how data is normalized helps you find the structure of the database faster.
 - You should be able to discuss with others (developers, DBA's) on how to best structure data
- As a developer:
 - Understanding database structure to be able to create ACID databases
 - Modelling data is a part of the development process



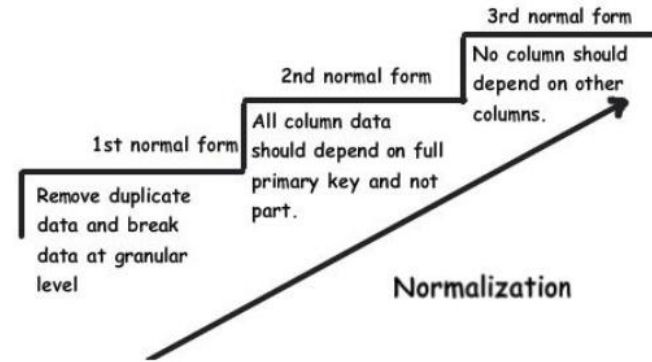
Atomicity
Consistency
Isolation
Durability

Process of normalization:

- After defining the problem (project: societal problem) and connecting the entities in your database:
- 1NF: First Normal Form
- 2NF: Second Normal Form
- 3NF: Third Normal Form
- BCNF: Boyce-Codd Normal Form
- 4NF: Fourth Normal Form
- ... etc. (more levels known, for the project, we go up to 3NF).

Process of normalization:

- After defining the problem (project: societal problem) and connecting the entities in your database:
- 1NF: First Normal Form
- 2NF: Second Normal Form
- 3NF: Third Normal Form
- BCNF: Boyce-Codd Normal Form
- 4NF: Fourth Normal Form
- ... etc. (more levels known, for the project, we go up to 3NF).



First Normal Form (1NF)

Simple explanation:

Each cell in the table should hold only one value. No lists or sets allowed.

Example:

If you've got a column for "Phone Numbers," don't jam multiple phone numbers into a single cell; instead, make a new row for each one.

First Normal Form (1NF)

If an entity contains a composite or multi-valued attribute, it violates 1NF

An entity is thus in 1NF if and only if:

1. It contains no composite values or multi-valued columns
2. It contains no repeating groups of attributes
3. The order in which data is stored does not matter

All of the data items in a column must mean the same thing.

First Normal Form (1NF)

If an entity contains a composite or multi-valued attribute, it violates 1NF

Name	Courses
Andre	KEN2210, KEN2110
Bella	KEN1292, KEN 2291
Yana	KEN1100
Andre	KEN2210

Name	Course1	Course2
Andre	KEN2210	KEN2110
Bella	KEN1292	KEN 2291
Yana	KEN1100	
Andre	KEN2210	

First Normal Form (1NF)

If an entity contains a composite or multi-valued attribute, it violates 1NF

Name	Courses
Andre	KEN2210, KEN2110
Bella	KEN1292, KEN 2291
Yana	KEN1100
Andre	KEN2210

Name	Course1	Course2
Andre	KEN2210	KEN2110
Bella	KEN1292	KEN 2291
Yana	KEN1100	
Andre	KEN2210	

ID	Name	Courses
1	Andre	KEN2210
1	Andre	KEN2110
2	Bella	KEN1292
2	Bella	KEN2291
3	Yana	KEN1100
4	Andre	KEN2210

The order in which data is stored does not matter / It contains no repeating groups of attributes

First Normal Form (1NF)

If an entity contains a composite or multi-valued attribute, it violates 1NF



Note that 'ID' is not the primary key in this Table (since it is not unique)



Name	Courses
Andre	KEN2210, KEN2110
Bella	KEN1292, KEN 2291
Yana	KEN1100
Andre	KEN2210



How does this last Table violate the 1NF principle? Resolve this issue ~2 min.



Name	Course1	Course2
Andre	KEN2210	KEN2110
Bella	KEN1292	KEN 2291
Yana	KEN1100	
Andre	KEN2210	



ID	Name	Courses
1	Andre	KEN2210
1	Andre	KEN2110
2	Bella	KEN1292
2	Bella	KEN2291
3	Yana	KEN1100
4	Andre	KEN2210

The order in which data is stored does not matter / It contains no repeating groups of attributes

First Normal Form (1NF)

If an entity contains a composite or multi-valued attribute, it violates 1NF



How does this last Table violate the 1NF principle? Resolve this issue
~2 min.

Student Table

student_id (PK)	name
1	Andre
2	Bella
3	Yana
4	Andre

Course Table

course_code (PK)
KEN2210
KEN2110
KEN1292

Enrollment Table (bridge or junction)

student_id (FK)	course_code (FK)
1	KEN2210
1	KEN2110
2	KEN1292

ID	Name	Courses
1	Andre	KEN2210
1	Andre	KEN2110
2	Bella	KEN1292
2	Bella	KEN2291
3	Yana	KEN1100
4	Andre	KEN2210



The order in which data is stored does not matter / It contains no repeating groups of attributes

Second Normal Form (2NF)

Simple explanation:

Make sure all the information in the table relates to the main thing you're talking about.

Example:

- In a "Student Courses" table, grades should depend on both the student and the course, not just one of them.
- Don't mix up unrelated things; a student's home address in this table; that's a different story.

Second Normal Form (2NF)

Any non-key attribute (i.e. attribute that is not part of any candidate key) should not be dependent on the proper subset of the table's candidate key.

An entity is thus in 2NF if and only if:

1. The data is in First Normal Form (1NF), and
2. Each non key field must reference the same thing as the primary key, and
3. Each table must contain data about only one type of thing

Second Normal Form (2NF)

Any non-key attribute (i.e. attribute that is not part of any candidate key) should not be dependent on the proper subset of the table's candidate key.



ID	First Name	Last Name	Marital Status	Course Title	Fee	Qualification	Grade
1	Kevin	Drumm	Single	Computer Science	£2000	Advanced Level	A
1	Kevin	Drumm	Single	Mathematics	£2500	Advanced Level	B
1	Kevin	Drumm	Single	Physics	£1800	Advanced Level	C
2	Murvin	Drake	Single	Physics	£1800	Advanced Level	B
2	Murvin	Drake	Single	Chemistry	£1800	Advanced Level	C
3	John	Jones	Single	Music	£1200	Diploma	C
4	Sally-Jane	Jones	Single	Biology	£1000	Certificate	A
4	Sally-Jane	Jones	Single	Economics	£1500	Diploma	B
5	David	Smith	Married	Mathematics	£2500	Advanced Level	C
5	David	Smith	Married	Physics	£1800	Advanced Level	D

Second Normal Form (2NF)

Any non-key attribute (i.e. attribute that is not part of any candidate key) should not be dependent on the proper subset of the table's candidate key.



ID	First Name	Last Name	Marital Status	Course Title	Fee	Qualification	Grade
1	Kevin	Drumm	Single	Computer Science	£2000	Advanced Level	A
1	Kevin	Drumm	Single	Mathematics	£2500	Advanced Level	B
1	Kevin	Drumm	Single	Physics	£1800	Advanced Level	C
2	Murvin	Drake	Single	Physics	£1800	Advanced Level	B
2	Murvin	Drake	Single	Chemistry	£1800	Advanced Level	C
3	John	Jones	Single	Music	£1200	Diploma	C
4	Sally-Jane	Jones	Single	Biology	£1000	Certificate	A
4	Sally-Jane	Jones	Single	Economics	£1500	Diploma	B
5	David	Smith	Married	Mathematics	£2500	Advanced Level	C
5	David	Smith	Married	Physics	£1800	Advanced Level	D

Second Normal Form (2NF)

Any non-key attribute (i.e. attribute that is not part of any candidate key) should not be dependent on the proper subset of the table's candidate key.

The primary key (PK) can be composed as a combination of ID and Course Title.

However, this key only describes which students are taking which courses.

It is not unique for each functional dependency; there are therefore partial dependencies.

ID	First Name	Last Name	Marital Status	Course Title	Fee	Qualification	Grade
1	Kevin	Drumm	Single	Computer Science	£2000	Advanced Level	A
1	Kevin	Drumm	Single	Mathematics	£2500	Advanced Level	B
1	Kevin	Drumm	Single	Physics	£1800	Advanced Level	C
2	Murvin	Drake	Single	Physics	£1800	Advanced Level	B
2	Murvin	Drake	Single	Chemistry	£1800	Advanced Level	C
3	John	Jones	Single	Music	£1200	Diploma	C
4	Sally-Jane	Jones	Single	Biology	£1000	Certificate	A
4	Sally-Jane	Jones	Single	Economics	£1500	Diploma	B
5	David	Smith	Married	Mathematics	£2500	Advanced Level	C
5	David	Smith	Married	Physics	£1800	Advanced Level	D

Second Normal Form (2NF)

Any non-key attribute (i.e. attribute that is not part of any candidate key) should not be dependent on the proper subset of the table's candidate key.

The primary key (PK) can be composed as a combination of ID and Course Title.

However, this key only describes which students are taking which courses.

It is not unique for each functional dependency; there are therefore partial dependencies.



How does this last Table violate the 2NF principle? Resolve this issue
~2 min.

ID	First Name	Last Name	Marital Status	Course Title	Fee	Qualification	Grade
1	Kevin	Drumm	Single	Computer Science	£2000	Advanced Level	A
1	Kevin	Drumm	Single	Mathematics	£2500	Advanced Level	B
1	Kevin	Drumm	Single	Physics	£1800	Advanced Level	C
2	Murvin	Drake	Single	Physics	£1800	Advanced Level	B
2	Murvin	Drake	Single	Chemistry	£1800	Advanced Level	C
3	John	Jones	Single	Music	£1200	Diploma	C
4	Sally-Jane	Jones	Single	Biology	£1000	Certificate	A
4	Sally-Jane	Jones	Single	Economics	£1500	Diploma	B
5	David	Smith	Married	Mathematics	£2500	Advanced Level	C
5	David	Smith	Married	Physics	£1800	Advanced Level	D

Second Normal Form (2NF)

Any non-key attribute (i.e. attribute that is not part of any candidate key) should not be dependent on the proper subset of the table's candidate key.

The primary key (PK) can be composed as a combination of ID and Course Title.

However, this key only describes which students are taking which courses.

It is not unique for each functional dependency; there are therefore partial dependencies.



How does this last Table violate the 2NF principle? Resolve this issue
~2 min.

Students

ID	First Name	Last Name	Marital Status
1	Kevin	Drumm	Single
2	Murvin	Drake	Single
3	John	Jones	Single
4	Sally-Jane	Jones	Single
5	David	Smith	Married

Courses

Course Title	Fee	Qualification
Computer Science	£2000	Advanced Level
Mathematics	£2500	Advanced Level
Physics	£1800	Advanced Level
Chemistry	£1800	Advanced Level
Music	£1200	Diploma
Biology	£1000	Certificate
Economics	£1500	Diploma

StudentCourse

ID	Course Title	Grade
1	Computer Science	A
1	Mathematics	B
1	Physics	C
2	Physics	B
2	Chemistry	C
3	Music	C
4	Biology	A
4	Economics	B
5	Mathematics	C
5	Physics	D

PK

Second Normal Form (2NF)

Any non-key attribute (i.e. attribute that is not part of any candidate key) should not be dependent on the proper subset of the table's candidate key.

The primary key (PK) can be composed as a combination of ID and Course Title.

However, this key only describes which students are taking which courses.

It is not unique for each functional dependency; there are therefore partial dependencies.



How does this last Table violate the 2NF principle? Resolve this issue ~2 min.

Students

ID	First Name	Last Name	Marital Status
1	Kevin	Drumm	Single
2	Murvin	Drake	Single
3	John	Jones	Single
4	Sally-Jane	Jones	Single
5	David	Smith	Married

Courses

Course Title	Fee	Qualification
Computer Science	£2000	Advanced Level
Mathematics	£2500	Advanced Level
Physics	£1800	Advanced Level
Chemistry	£1800	Advanced Level
Music	£1200	Diploma
Biology	£1000	Certificate
Economics	£1500	Diploma

StudentCourse

ID	Course Title	Grade
1	Computer Science	A
1	Mathematics	B
1	Physics	C
2	Physics	B
2	Chemistry	C
3	Music	C
4	Biology	A
4	Economics	B
5	Mathematics	C
5	Physics	D

PK



StudentCourse entity uses a unique combination of attribute 'ID' and 'Course Title' as Compound Primary Key

Third Normal Form (3NF)

Simple explanation:

- Each column should tell you something directly about the key.
- No column should tell you something that you could already find out through another column.

Example:

- Suppose we store employees in one table:
 - EmployeeID $\rightarrow$ DepartmentID
 - DepartmentID $\rightarrow$ DepartmentName
- Here EmployeeID $\rightarrow$ DepartmentName would add a transitive dependency (through DepartmentID). That breaks 3NF.

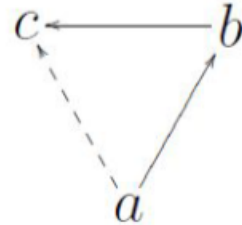
Transitive Dependency: $X \rightarrow Z$ and $Z \rightarrow Y \Rightarrow X \rightarrow Y$.

Third Normal Form (3NF)

In the third normal form, the following conditions are required:

1. It is in 2NF
2. No non-key attribute depends on another non-key attribute

i.e. there is no other non-key attribute that you would need to change in a table if you changed a non-key attribute



Transitive Dependency: $X \rightarrow Z$ and $Z \rightarrow Y \Rightarrow X \rightarrow Y$.

Third Normal Form (3NF)



Course Title	Fee	Qualification	Teacher ID	Teacher Name
Computer Science	£2000	Advanced Level	1	Miss Lovelace
Mathematics	£2500	Advanced Level	2	Mr Pascal
Physics	£1800	Advanced Level	3	Mr Einstein
Chemistry	£1800	Advanced Level	4	Mr Bunsen
Music	£1200	Diploma	5	Miss Holiday
Biology	£1000	Certificate	6	Mr Darwin
Economics	£1500	Diploma	7	Mr Keynes

Transitive functional dependencies:

CourseTitle $\rightarrow$ TeacherID

TeacherID $\rightarrow$ TeacherName

So: CourseTitle $\rightarrow$ TeacherName
(transitive dependency )

Third Normal Form (3NF)

Course Title	Fee	Qualification	Teacher ID	Teacher Name
Computer Science	£2000	Advanced Level	1	Miss Lovelace
Mathematics	£2500	Advanced Level	2	Mr Pascal
Physics	£1800	Advanced Level	3	Mr Einstein
Chemistry	£1800	Advanced Level	4	Mr Bunsen
Music	£1200	Diploma	5	Miss Holiday
Biology	£1000	Certificate	6	Mr Darwin
Economics	£1500	Diploma	7	Mr Keynes



Transitive functional dependencies:

CourseTitle $\rightarrow$ TeacherID

TeacherID $\rightarrow$ TeacherName

So: CourseTitle $\rightarrow$ TeacherName
(transitive dependency )



How does this Table violate the 2NF principle?

Third Normal Form (3NF)



Course Title	Fee	Qualification	Teacher ID	Teacher Name
Computer Science	£2000	Advanced Level	1	Miss Lovelace
Mathematics	£2500	Advanced Level	2	Mr Pascal
Physics	£1800	Advanced Level	3	Mr Einstein
Chemistry	£1800	Advanced Level	4	Mr Bunsen
Music	£1200	Diploma	5	Miss Holiday
Biology	£1000	Certificate	6	Mr Darwin
Economics	£1500	Diploma	7	Mr Keynes

TeacherName depends on TeacherID, not on CourseTitle

→ Causes **update anomalies**:
if Miss Lovelace changes their name, you must update every course they teach.

Fix: split in 2 columns (fixes both 2NF and 3NF issues)



How does this Table violate the 2NF principle?

Third Normal Form (3NF): another example

Order ID (PK)	Customer ID	Customer City	Delivery Cost
1001	42	Amsterdam	£15
1002	87	Rotterdam	£18
1003	42	Amsterdam	£15

Transitive functional dependencies:

Order ID → Customer ID → Customer City → Delivery Cost

- Customer City depends on Customer ID, not on Order ID
 - Delivery Cost depends on Customer City, not on Order ID
- (both are transitive dependencies ❌)

Third Normal Form (3NF): another example

Order ID (PK)	Customer ID	Customer City	Delivery Cost
1001	42	Amsterdam	£15
1002	87	Rotterdam	£18
1003	42	Amsterdam	£15

Transitive functional dependencies:

Order ID → Customer ID → Customer City → Delivery Cost

- Customer City depends on Customer ID, not on Order ID
 - Delivery Cost depends on Customer City, not on Order ID
- (both are transitive dependencies ❌)



Row 1 and row 3 both store Amsterdam and £15 for customer 42. If the delivery cost for Amsterdam changes, you must update multiple rows — and if you miss one, the database becomes inconsistent.

Third Normal Form (3NF): another example

Order ID (PK)	Customer ID	Customer City	Delivery Cost
1001	42	Amsterdam	£15
1002	87	Rotterdam	£18
1003	42	Amsterdam	£15

When normalizing to 3NF, look for categories or identifiers that suggest “facts about facts.”

Third Normal Form (3NF): another example

Order ID (PK)	Customer ID	Customer City	Delivery Cost
1001	42	Amsterdam	£15
1002	87	Rotterdam	£18
1003	42	Amsterdam	£15



Fix this Table, to meet 3NF

When normalizing to 3NF, look for categories or identifiers that suggest “facts about facts.”

Third Normal Form (3NF): another example

Order ID (PK)	Customer ID	Customer City	Delivery Cost
1001	42	Amsterdam	£15
1002	87	Rotterdam	£18
1003	42	Amsterdam	£15



Fix this Table, to meet 3NF



Two or three Tables?

Order Entity

Order ID (PK)	Customer ID (FK)
1001	42
1002	87
1003	42

Customer Entity

Customer ID (PK)	Customer City	Delivery Cost
42	Amsterdam	£15
87	Rotterdam	£18

Splitting in two Tables (entities) retains another transitive dependency:
Customer ID → Customer City → Delivery Cost
However, Delivery cost does not depend on Customer ID

Third Normal Form (3NF): another example

Order ID (PK)	Customer ID	Customer City	Delivery Cost
1001	42	Amsterdam	£15
1002	87	Rotterdam	£18
1003	42	Amsterdam	£15



Fix this Table, to meet 3NF



Two or three Tables?

Order Entity

Order ID (PK)	Customer ID (FK)
1001	42
1002	87
1003	42

Customer Entity

Customer ID (PK)	Customer City
42	Amsterdam
87	Rotterdam

City Entity

Customer City (PK)	Delivery Cost
Amsterdam	£15
Rotterdam	£18

Boyce-Codd Normal Form (BCNF)

Simple explanation:

- 3NF: Every column should be a fact about the key
 - BCNF goes further: even a key attribute cannot determine something unless it's the whole key
1. It is already in 3NF
 2. For every non-trivial dependency $X \rightarrow Y$, X is a superkey

Key point

- Only candidate keys can determine other attributes
- No attribute (prime or non-prime) should depend on a non-key

Boyce-Codd Normal Form (BCNF)

Definition:

A relation is in BCNF if, for every non-trivial functional dependency $X \rightarrow Y$,
👉 X is a superkey of the relation.

Key type	Definition
Superkey	Any set of columns that uniquely identifies a row — may contain redundant columns
Candidate key	A minimal superkey — no column can be removed and still guarantee uniqueness → the smallest set of columns that uniquely identifies a row
Primary key	The chosen candidate key used to identify rows in the table



- Prime attribute: any column that appears in any of the candidate keys
- Non-prime attribute: any column that does not appear in any candidate key

Boyce-Codd Normal Form (BCNF)

Definition:

A relation is in BCNF if, for every non-trivial functional dependency $X \rightarrow Y$,

👉 X is a superkey of the relation.

- Trivial FD: $X \rightarrow Y$ is trivial if $Y \subseteq X$.
 - Example: $\{\text{StudentID}, \text{CourseID}\} \rightarrow \text{StudentID}$ is trivial, since StudentID is part of the left-hand side.
- Non-trivial FD: $X \rightarrow Y$ where Y is not a subset of X .
 - Example: $\text{StudentID} \rightarrow \text{Name}$

Boyce-Codd Normal Form (BCNF)

Definition:

A relation is in BCNF if, for every non-trivial functional dependency $X \rightarrow Y$,

👉 X is a superkey of the relation.

- Trivial FD: $X \rightarrow Y$ is trivial if $Y \subseteq X$.
 - Example: $\{\text{StudentID}, \text{CourseID}\} \rightarrow \text{StudentID}$ is trivial, since StudentID is part of the left-hand side.
 - Non-trivial FD: $X \rightarrow Y$ where Y is not a subset of X .
 - Example: $\text{StudentID} \rightarrow \text{Name}$
- 
- A **trivial** functional dependency (FD) is one that tells you nothing new.
 - A **non-trivial** FD is one that actually *says something* about your data $\rightarrow$ and can potentially be violated.
 - Example: “If I know your Student ID, do I know your Name?”

Boyce-Codd Normal Form (BCNF)

Definition:

A relation is in BCNF if, for every non-trivial functional dependency $X \rightarrow Y$, 🙌 X is a superkey of the relation.

Student Id	Subject	Professor
2394	Java	Smirnov
2030	C++	Hortal
2455	Databases	Pepels
6472	Cryptography	Not Pepels



$\{\text{Student Id, Subject}\} \rightarrow \text{Professor}$

$\text{Subject} \rightarrow \text{Professor}$

but also:

$\text{Professor} \rightarrow \text{Subject}$



3NF: Non-prime attributes should not depend on a non-prime attribute

BCNF: Any attribute (prime or nonprime) cannot depend on non-superkey determinants

Boyce-Codd Normal Form (BCNF)

Definition:

A relation is in BCNF if, for every non-trivial functional dependency $X \rightarrow Y$, 🙌 X is a superkey of the relation.



Student Id	Subject	Professor
2394	Java	Smirnov
2030	C++	Hortal
2455	Databases	Pepels
6472	Cryptography	Not Pepels



Two candidate keys exist:

- {Student ID, Subject} — uniquely identifies a row
- {Student ID, Professor} — also uniquely identifies a row

Student ID, Subject, and Professor are all prime attributes

Therefore, Table is valid for 3NF

Boyce-Codd Normal Form (BCNF)

Definition:

A relation is in BCNF if, for every non-trivial functional dependency $X \rightarrow Y$, 🙌 X is a superkey of the relation.



But, there is another dependency in this data:

Subject (or course) $\rightarrow$ Professor (or teacher)

If we know the course subject, do we know the name of the teacher?

Answer: No

Therefore, **non-trivial** FD

What if prof. Pepels stops teaching Databases and is replaced by Slenter?
We need to update each row where the course Subject is Databases.
Missing one means inconsistency



Student Id	Subject	Professor
2394	Java	Smirnov
2030	C++	Hortal
2455	Databases	Pepels
6472	Cryptography	Not Pepels

Boyce-Codd Normal Form (BCNF)

Definition:

A relation is in BCNF if, for every non-trivial functional dependency $X \rightarrow Y$, 🙌 X is a superkey of the relation.

Fix: split in Two Tables



Student Id	Course Id
2394	1
2030	2
2455	3
6472	4



Course Id	Subject	Professor
1	Java	Smirnov
2	C++	Hortal
3	Databases	Pepels
4	Cryptography	Not Pepels

Fourth Normal Form (4NF)

Simple explanation:

Imagine you're making a profile of John.

- John likes Chess and Swimming.
- John speaks English and Dutch.

Person	Hobby	Language
John	Chess	English
John	Chess	Dutch
John	Swimming	English
John	Swimming	Dutch

But now it looks like every hobby is tied to every language: as if “Chess = English” and “Swimming = Dutch”. That’s not true.

👉 Don't mix two independent lists in one table.
Keep hobbies and languages in separate tables so each fact stays clean.

Fourth Normal Form (4NF)

1. Table is in BCNF2.
2. For every non-trivial multivalued dependency $X \twoheadrightarrow Y$: 🙌 X is a superkey (a candidate key or a superset of one).

MVD ($A \twoheadrightarrow B$) : **Multi-Valued Dependency**

- For one value of A , there can be many values of B
- Independent of other attributes

A	B	C
A1	B1	C1
A1	B2	C2

🙌 Don't store two independent lists in the same table.

Fourth Normal Form (4NF)



Student Id	Subject	Hobby
2394	Java	Tennis
2394	C++	Knitting
2455	Databases	Jazz
2455	Cryptography	Eating Meat

Keep relations independent, which means, store them in separate tables.



Student Id	Subject
2394	Java
2394	C++
2455	Python
2455	Cryptography



Student Id	Hobby
2394	Tennis
2394	Knitting
2455	Jazz
2455	Eating Meat

Fourth Normal Form (4NF)



Fix this Table, to meet 4NF (~2 min).

MusicianID	Instrument	Genre
1	Guitar	Rock
1	Guitar	Jazz
1	Piano	Rock
1	Piano	Jazz
2	Drums	Pop
2	Drums	Blues
2	Bass	Pop
2	Bass	Blues



Fourth Normal Form (4NF)



Fix this Table, to meet 4NF (~2 min).

MusicianID	Instrument	Genre
1	Guitar	Rock
1	Guitar	Jazz
1	Piano	Rock
1	Piano	Jazz
2	Drums	Pop
2	Drums	Blues
2	Bass	Pop
2	Bass	Blues



MusicianID	Instrument
1	Guitar
1	Piano
2	Drums
2	Bass



MusicianID	Genre
1	Rock
1	Jazz
2	Pop
2	Blues



MVDs have been separated into two distinct relations:

1. one for the instruments a musician plays and
2. another for the genres they perform in

Selfstudy

A Longer Example on Database Normalization:

- <https://www.youtube.com/watch?v=I5DCnCzDb8g>
- A.Beaulieu, Learning SQP, 2020 (3rd edition, O'Reilly:
 - Chapter 2: Creating and Populating a Database

Questions?



See you tomorrow

Tutorial and Assignment 2 will be released today!

